HBALS: A Hybrid Bees Algorithm with Adaptive Local Search for the Traveling Salesman Problem (TSP)

Erfan Khadiv
Department of Computer and Information Technology, Islamic Azad University, Ardabil Branch, Ardabil, Iran
Corresponding author: erfan.khadiv.dev@gmail.com

Code and experimental data:
https://github.com/ErfanKhadiv/HBALS-TSP

Acknowledgments

The author thanks Dr. Masoud Bekravi for serving as the supervisor of the Computer Project course at Islamic Azad University, Ardabil, under which this work was carried out.

The author used ChatGPT (OpenAI) and Claude (Anthropic) for language editing and for feedback on the manuscript and repository documentation. The author reviewed and edited all content and takes full responsibility for the work.

Abstract

The Traveling Salesman Problem (TSP) is a classical combinatorial optimization problem for which metaheuristic methods are commonly used to obtain high-quality solutions within practical computational budgets. The Bees Algorithm (BA) provides a population-based search mechanism that combines exploration and exploitation, but its performance can be limited by the quality of local search and neighborhood selection. This study proposes HBALS (Hybrid Bees Algorithm with Adaptive Local Search), a hybrid variant of the Bees Algorithm that combines mixed Nearest-Neighbor and random initialization, elite- and selected-site search, adaptive neighborhood adjustment, and lightweight local-search operators. The local-search component primarily uses 2-opt, with occasional lightweight 3-opt-style refinement. HBALS was evaluated on six Traveling Salesman Problem Library (TSPLIB) benchmark instances with up to 195 cities using 20 independent runs and 300 iterations per run. The method was also compared with implementations of BA, Ant Colony Optimization (ACO), Genetic Algorithm (GA), and Particle Swarm Optimization (PSO) under the experimental configuration used in this study. HBALS reached the best-known solution on eil51, berlin52, and st70, while producing solutions close to the best-

known values on the larger tested instances. Across the six tested instances, HBALS reduced the best-tour length by 15.3% to 68.0% relative to the implemented BA baseline, with an arithmetic mean reduction of 41.5%. The results indicate that the tested hybrid configuration can improve solution quality over the implemented BA baseline, although performance varies across problem instances and additional evaluation on larger benchmark sets is required.

Keywords: Combinatorial optimization, Traveling Salesman Problem (TSP), Bees Algorithm, local search, metaheuristic algorithms

1. Introduction

Optimization problems are among the most fundamental research areas in engineering, management, and data science, since their objective is to find the best possible configuration for a set of variables. Such problems are generally divided into continuous and discrete categories, and a significant portion of discrete problems fall under “combinatorial optimization,” where the objective is to select an element from a finite or countable set — which may be an integer, a subset, a permutation, or a graph structure [1]. Well-known examples in this category include the Traveling Salesman Problem (TSP), the Quadratic Assignment Problem (QAP), and scheduling problems. Although exact methods guarantee an optimal solution for any bounded problem, for NP-Hard problems the exponential growth of the search space typically makes such methods computationally impractical; consequently, metaheuristic and approximate methods have gained particular importance over recent decades [1].

In recent years, the Bees Algorithm and its improved variants have received considerable attention. One of the most recent studies introduced a “single-parameter” version of the algorithm that reduces the complexity of parameter tuning while increasing search efficiency, demonstrating notable performance on complex problems such as the TSP [2].

Beyond the growing importance of the Bees Algorithm in solving complex routing problems, combining it with local-search operators is a common way to improve solution quality. Operators such as 2-opt and 3-opt are widely used to refine TSP tours and reduce tour length. The proposed method, HBALS (Hybrid Bees Algorithm with Adaptive Local Search), uses 2-opt together with a lightweight 3-opt-style refinement rather than a full 3-opt neighborhood.

Based on these observations, the present study investigates whether combining local-search refinement with adaptive neighborhood control can improve the performance of a Bees-Algorithm-based method on the selected TSP instances.

2. Related Work

2.1 Review of Related Research

The Traveling Salesman Problem, as one of the best-known NP-hard problems, has consistently attracted the attention of optimization researchers. In recent years, the Bees Algorithm and its improved variants have been a central focus of research in this area.

In a comparative study, Hartono et al. (2024) examined several reduced-parameter variants of the Bees Algorithm applied to the TSP. Their results show that simplifying the number of parameters not only reduces tuning complexity but also preserves competitive performance on standard benchmark datasets [4].

In another study, Sahin (2023) proposed a hybrid approach combining the Bees Algorithm with the Nearest Neighbor method to improve the quality of the initial solution. This combination accelerated convergence and reduced performance variance across several TSP instances [3].

Similarly, Suluova & Pham (2024) introduced a variant with only a single control parameter, aiming to reduce the tuning difficulty found in the classical version. Their results indicate that removing multiple parameters can simplify implementation while keeping the algorithm's performance at a competitive level [2].

Overall, the literature review shows that most contemporary approaches focus on simplifying the algorithm's structure, reducing dependence on numerous parameters, and strengthening either local search steps or constructive methods. In this context, the proposed HBALS algorithm is also designed — through adaptive local search and adaptive neighborhood control — to address the limitations of earlier Bees-Algorithm variants.

2.2 Rationale for Selecting the Bees Algorithm

The Bees Algorithm (BA) was selected as the base method because its population-based structure provides a natural framework for combining global exploration with intensified search around promising solutions. In the standard formulation, a set of scout bees explores the solution space, while additional search effort is allocated to selected and elite sites according to solution quality [5,6]. This structure is also suitable for hybridization with problem-specific local-search operators.

For the TSP, a candidate solution can be represented directly as a permutation of cities, allowing neighborhood-based modifications such as swap operations and 2-opt moves to be incorporated without introducing a separate continuous-to-discrete transformation. The present study therefore focuses on improving the local exploitation component of BA rather than proposing a new population representation.

The objective of HBALS is consequently not to establish that BA is universally superior to other metaheuristics, but to investigate whether adaptive neighborhood control and local-search

refinement can improve the performance of a BA-based method on the selected TSP benchmark instances.

3. The Bees Algorithm

3.1 Overview

The Bees Algorithm is a population-based metaheuristic developed for solving both continuous and combinatorial optimization problems. It is inspired by the natural foraging behavior of honeybees searching for nectar, and it simulates the process of locating food sources and sharing information among bees within the hive [5, 6]. It belongs to the broader family of Swarm Intelligence algorithms, which draw inspiration from the collective behavior of organisms in nature.

3.2 Natural Bee Foraging Behavior

In a honeybee colony, roughly one-quarter of the members act as forager bees, whose primary task is to locate and collect nectar, pollen, water, and resin from the environment surrounding the hive [7]. Among these forager bees, only a relatively small fraction — approximately 5% to 25% — act as scout bees [8].

During the search and foraging process, scout bees move randomly among flower patches to identify new nectar sources [9]. After finding a suitable source, a scout bee returns to the hive, unloads the nectar, and performs the waggle dance to inform other bees of the source's location, distance, and quality [6]. Following this dance, a group of follower bees is recruited from within the hive and, together with the scout bee, moves toward the source. The number of bees recruited to a given source depends on its quality: richer sources that require less effort to exploit attract more bees [9]. The duration and intensity of the dance movements are also directly related to the source's distance from the hive [10].

3.3 Structure and Parameters of the Bees Algorithm

The Bees Algorithm uses a set of scout bees to sample the solution space. Each bee in this process represents a possible solution to the optimization problem. The algorithm requires initializing a set of parameters at the start of execution, including [6, 9, 11]:

n_b — the number of scout bees

m — the number of selected sites among the visited points

e — the number of elite sites among the top m sites

nep — the number of bees recruited to each elite site

nsp — the number of bees recruited to the remaining (non-elite) selected sites

ngh — the size of the search neighborhood (patch size) around each site

Stopping criterion — an error threshold or a maximum number of iterations

4. Modeling the TSP for the Bees Algorithm

4.1 Definition of the Traveling Salesman Problem

The Traveling Salesman Problem (TSP) is one of the most widely studied problems in combinatorial optimization. Its first mathematical formulation is generally credited to Merrill Flood in 1948, developed while studying school-bus routing. The problem considers a set of cities and the paths between them, with the objective of finding the shortest possible route such that a salesman starts at one city, visits every other city exactly once, and returns to the starting point.

The objective is to find a permutation of the cities that minimizes the total length of the closed tour:

$$L(\pi) = \sum_{i=1}^{N-1} d_{\pi_i,\pi_{i+1}} + d_{\pi_N,\pi_1}$$

where N is the number of cities, π denotes a permutation representing a candidate tour, and d_{ij} is the distance between cities i and j.

The TSP can be modeled as an undirected graph in which each vertex represents a city and each edge represents the path between two cities. The weight of each edge typically represents the distance or cost of travel between the two cities, and the objective is to find the route that minimizes the total cost [12].

Computationally, the problem belongs to the NP-Hard class, and its search space grows exponentially as the number of cities increases. As a result, finding an exact solution for large instances is extremely time-consuming or even impractical [3, 4]. Under these conditions, heuristic and metaheuristic algorithms — such as evolutionary algorithms and swarm intelligence methods — are proposed as efficient approaches for obtaining near-optimal solutions [3, 4].

Due to its wide range of real-world applications, the TSP has consistently attracted research attention. Its applications include transportation, logistics, scheduling, industrial manufacturing, tourism, and even DNA sequencing [4]. For this reason, it is widely used as a standard benchmark for evaluating the performance of optimization algorithms [4].

4.2 Adapting the Bees Algorithm for the TSP

When applying this algorithm to the TSP, each bee represents a possible route among the cities. In the initial stage, a population of random routes is generated, and candidate tours are evaluated according to their tour length, and search effort is concentrated around promising solutions. Worker bees then continue searching around the best routes (local neighborhoods) to achieve incremental

improvements [4]. This mechanism creates a balance between exploration and exploitation of the search space, which is essential for avoiding entrapment in local optima [13].

5. Description of the Proposed HBALS Algorithm

HBALS is a hybrid variant of the Bees Algorithm designed for the TSP. Its main modification is the integration of adaptive neighborhood control and local-search refinement into the site-based search process. The algorithm maintains a population of candidate tours, ranks them according to tour length, intensifies the search around elite sites, applies lighter search to additional selected sites, and fills the remaining population with newly generated scout solutions.

5.1 HBALS Search Procedure

Step 1 — Population initialization

An initial population of $n_b = 20$ tours is generated using a mixture of Nearest-Neighbor and random construction. With nn_ratio = 0.5, half of the initial tours are constructed using the Nearest-Neighbor heuristic from different starting cities, while the remaining tours are generated as random permutations.

Step 2 — Adaptive neighborhood adjustment

At the beginning of each iteration, the neighborhood size is adjusted according to the number of consecutive iterations without improvement. If fewer than three consecutive non-improving iterations have occurred, the neighborhood size is reduced using the factor $\alpha = 0.9$, subject to a minimum value of 1. If three or more consecutive non-improving iterations have occurred, the neighborhood size is increased using the factor $\beta = 1.05$, subject to an upper bound of $N/2$.

Step 3 — Evaluation and ranking

The tour length of every candidate is evaluated using the TSP objective function, and the population is ranked from the shortest to the longest tour.

Step 4 — Elite-site search

The best $e = 3$ ranked solutions are selected as elite sites. For each elite solution, $nep = 8$ candidate tours are generated. Each candidate is first perturbed using a number of random swaps determined by the current neighborhood size. A 2-opt local-search procedure is then applied with probability 0.7. With probability 0.1, an additional lightweight 3-opt-style refinement is applied.

Step 5 — Selected-site search

The first $m = 12$ ranked solutions form the selected-site set. After removing the three elite sites, each remaining selected site is explored using $nsp = 4$ candidate solutions. These candidates receive a lighter perturbation and are subjected to 2-opt refinement with probability 0.3.

Step 6 — Scout-bee exploration

After elite- and selected-site search, new random tours are generated until the population size returns to $n_b = 20$. This step preserves exploration and introduces new regions of the search space.

Step 7 — Global-best update and termination

The best solution in the newly generated population is compared with the global best solution found so far. If an improvement occurs, the non-improvement counter is reset; otherwise, the counter is increased. The search terminates when either 300 maximum iterations are reached or the number of consecutive non-improving iterations reaches the configured threshold of 20.

Step 8 — Final solution

The best tour recorded during the entire run is returned as the final solution.

Algorithm 1. Pseudocode of HBALS for the TSP

```
Input:
    TSP distance matrix D
    Population size n_b
    Number of selected sites m
    Number of elite sites e
    Elite-site recruitment nep
    Selected-site recruitment nsp
    Initial neighborhood size ngh
    Neighborhood shrink factor α
    Neighborhood expansion factor β
    Maximum number of iterations T
    Non-improvement threshold H

Output:
    Best tour found

1:  Initialize the population using Nearest-Neighbor and random construction
2:  Evaluate the tour length of every solution
3:  Set the best solution as the global best
4:  Set current_ngh ← ngh
5:  Set no_improve_count ← 0

6:  for iteration = 1 to T do

7:      if no_improve_count ≥ 3 then
8:          current_ngh ← min(int(current_ngh × β), floor(N / 2))
9:      else
10:         current_ngh ← max(int(current_ngh × α), 1)
```

```
    end if

    Rank all solutions according to tour length
    Select the best e solutions as elite sites
    Select the best m solutions as selected sites

    for each elite site do
        Generate nep candidate solutions around the elite site
        Apply current_ngh random-swap perturbations
        Apply 2-opt with probability 0.7
        Apply lightweight 3-opt-style refinement with probability 0.1
    end for

    for each non-elite selected site do
        Generate nsp candidate solutions around the selected site
        Apply max(1, floor(current_ngh / 2)) random-swap perturbations
        Apply 2-opt with probability 0.3
    end for

    Generate random scout solutions until the population size reaches n_b
    Evaluate the new population

    Determine the best solution in the new population

    if the new best solution improves the global best then
        Update the global best solution
        no_improve_count ← 0
    else
        no_improve_count ← no_improve_count + 1
    end if

    if no_improve_count ≥ H then
        break
    end if

end for

return global best solution
```

6. Experimental Results

6.1 Experimental Environment

All experiments were conducted on a computer running Windows 11 with a 12th-generation Intel Core i3 processor and 16 GB of RAM. The algorithms were implemented in Python 3.12.7. The convergence plots (Figs. 1–6) were generated using the Matplotlib library for Python. All reported experiments used the same computing environment for consistency of runtime measurements. The six benchmark instances (eil51, berlin52, st70, kroA100, ch150, and rat195) were obtained from the Traveling Salesman Problem Library (TSPLIB) [14]. Distances follow the TSPLIB convention, with Euclidean distances rounded to the nearest integer.

Use of AI-assisted tools. ChatGPT (OpenAI) and Claude (Anthropic) were used for language editing and for feedback on the manuscript and repository documentation. The author reviewed and edited the resulting material and takes full responsibility for the final content.

6.2 HBALS Parameters and Rationale

To evaluate the performance of the proposed hybrid Bees Algorithm (HBALS), the **Fast** configuration was selected, prioritizing computational efficiency while maintaining acceptable solution quality. Table 1 presents the parameter values used in the experiments along with the rationale for their selection.

Table 1. Selected parameters and configuration of the HBALS fast preset

Parameter	Value	Description / Rationale
n_b	20	Number of solutions maintained in the population
m	12	Number of selected sites
e	3	Number of elite sites
nep	8	Number of candidate solutions generated per elite site
nsp	4	Number of candidate solutions generated per non-elite selected site
ngh	3	Initial number of random-swap perturbations used to construct a neighborhood
α	0.9	Factor used to reduce the neighborhood size when fewer than three consecutive non-improving iterations have occurred
β	1.05	Factor used to increase the neighborhood size after three or more consecutive non-improving iterations
nn_ratio	0.5	Fraction of the initial population generated using Nearest-Neighbor construction
no_improve_threshold	20	Number of consecutive non-improving iterations allowed before early termination

The reported experiments use the fast HBALS preset. The initial population contains 20 solutions, of which 50% are generated using the Nearest-Neighbor heuristic and the remainder are generated randomly. The best $e = 3$ solutions are treated as elite sites, while the first $m = 12$ ranked solutions define the selected-site set. Elite sites receive $nep = 8$ neighborhood candidates, whereas the remaining selected sites receive $nsp = 4$ candidates each.

The neighborhood size is controlled adaptively during the search. If fewer than three consecutive non-improving iterations have occurred, the current neighborhood size is reduced by the factor $\alpha = 0.9$. After three or more consecutive non-improving iterations, it is increased by the factor $\beta = 1.05$, subject to an upper bound of $N/2$. The search terminates early when the number of consecutive non-improving iterations reaches 20. No automated parameter optimization was performed; the reported values correspond to the manually specified fast preset.

2-opt is applied as a first-improvement search limited to 20 improving moves per candidate at elite sites and 10 at selected sites. The lightweight 3-opt-style step is applied only at elite sites; it evaluates 3 random segment reversals and keeps the best.

Because the neighborhood size is updated with integer truncation, it decreases from 3 to 1 within the first two iterations and cannot increase from 1 with β = 1.05. The perturbation strength therefore remains at one random swap for the rest of each run, and the adaptive expansion step is effectively inactive on the tested instances.

6.3 Number of Runs and Iterations

Each algorithm was evaluated using 20 independent runs with a maximum of 300 iterations per run. The number of runs was selected to provide repeated observations of the stochastic behavior of the methods and to allow estimation of the mean and standard deviation of the best tour length obtained in each run. A maximum of 300 iterations was used as a common computational budget across the tested methods.

6.4 HBALS Performance Results

Table 2 reports the best, mean and standard deviation of the tour length, the mean gap from the best-known solution (BKS), and the average runtime obtained by HBALS on the six benchmark instances.

Table 2. Performance results of HBALS across TSP benchmark instances

Dataset	N (cities)	BKS	Best	Mean	Std. Dev.	Mean Gap (%)	Time (s)
eil51	51	426	426	427.85	1.68	0.43	4.62
berlin52	52	7542	7542	7543	2.00	0.01	5.31
st70	70	675	675	679.4	2.82	0.65	11.91

Dataset	N (cities)	BKS	Best	Mean	Std. Dev.	Mean Gap (%)	Time (s)
kroA100	100	21282	21296	21388.6	79.06	0.50	39.17
ch150	150	6528	6570	6627.95	40.13	1.53	83.68
rat195	195	2323	2349	2375.3	14.16	2.25	161.51

The mean gap with respect to the best-known solution (BKS) was calculated as:

$$Gap_{mean}(\%) = \frac{Mean - BKS}{BKS} \times 100$$

where $Mean$ denotes the average best tour length obtained over the independent runs. A smaller value indicates a result closer to the best-known solution.

The mean gap from the BKS ranges from 0.01% to 2.25% across the tested instances. Runtime increases with instance size, ranging from 4.62 s on eil51 to 161.51 s on rat195.

6.5 Comparison of HBALS with Baseline Metaheuristics

6.5.1 Baseline Parameter Settings

The standard Bees Algorithm (BA) baseline uses the same population and site parameters as the HBALS fast configuration, while excluding the HBALS-specific adaptive neighborhood control and local-search refinement. The BA implementation uses fully random population initialization, elite- and selected-site recruitment, and a fixed neighborhood size of $ngh = 3$. For each recruited bee, three independent random single-swap neighbors of the site's base tour are evaluated, and the best candidate found is retained. Scout bees generate new random solutions to maintain exploration. Ant Colony Optimization (ACO), Genetic Algorithm (GA), and Particle Swarm Optimization (PSO) use the parameter settings specified in Table 3. These settings correspond to the implementations available in the accompanying code repository.

Table 3. Parameter settings of the baseline algorithms

Algorithm	Population / Agents	Main parameters
BA	20	$m = 12, e = 3, nep = 8, nsp = 4, ngh = 3$
ACO	20 ants	$\alpha = 1.0, \beta = 2.0, \rho = 0.5, Q = 100, \tau_0 = 0.1$
GA	20	Crossover rate = 0.8; Mutation rate = 0.1; Tournament size = 3; Elitism = 2
PSO	20	Inertia weight = 0.7; Cognitive coefficient = 0.8; Social coefficient = 0.9; Mutation rate = 0.1; $nn_ratio = 0.3$

Note: — $\boldsymbol{ngh}$: number of independent one-swap neighborhood candidates evaluated for each recruited bee.

The baseline methods were not configured with identical internal parameters because each algorithm uses a different search mechanism. Instead, the experiments used the parameter configurations implemented in the accompanying repository, while keeping the number of independent runs and the maximum iteration budget consistent across methods. Therefore, the

comparison evaluates the performance of the implemented configurations rather than claiming equivalence of all algorithmic computational operations.

6.5.2 Performance Comparison

Table 4 presents the performance comparison of HBALS with the four implemented baseline algorithms across the six TSP benchmark instances. The reported metrics include the best solution found across the independent runs, the mean best-tour length, standard deviation, and average execution time.

Table 4. Performance comparison of HBALS and baseline metaheuristic algorithms across TSP benchmark instances

Dataset	Algorithm	Best	Mean	Std	Time (s)
eil51	HBALS	426	427.85	1.68	4.62
eil51	BA	503	543.70	19.22	1.77
eil51	GA	674	770.65	45.8	0.34
eil51	ACO	438	452.75	6.27	20.66
eil51	PSO	489	505.65	3.96	0.29
berlin52	HBALS	7542	7543	2.00	5.31
berlin52	BA	9592	9838.55	201.63	1.70
berlin52	GA	12368	14009	908.49	0.33
berlin52	ACO	7548	7684.45	57.57	21.97
berlin52	PSO	8801	8970.65	38.93	0.34
st70	HBALS	675	679.4	2.82	11.91
st70	BA	978	1084.05	58.10	1.81
st70	GA	1570	1711.2	99.64	0.47
st70	ACO	698	731.6	13.63	45.94
st70	PSO	765	810.9	10.76	0.35
kroA100	HBALS	21296	21388.6	79.06	39.17
kroA100	BA	44975	48562.60	2445.02	1.92
kroA100	GA	71781	79111.4	3629.83	0.67
kroA100	ACO	23240	23611.4	230.5	123.28
kroA100	PSO	25816	26117.15	69.08	0.5
ch150	HBALS	6570	6627.95	40.13	83.68
ch150	BA	16817	18193.60	677.25	2.52
ch150	GA	27083	29230.7	901.6	1.23

Dataset	Algorithm	Best	Mean	Std	Time (s)
ch150	ACO	6758	6964.35	125.26	363.54
ch150	PSO	7309	7309	0.0	0.68
rat195	HBALS	2349	2375.3	14.16	161.51
rat195	BA	7344	7843.50	221.45	2.59
rat195	GA	11063	12062.65	548.17	1.95
rat195	ACO	2466	2552.95	42.5	767.89
rat195	PSO	2742	2751.2	2.48	0.93

6.5.3 Comparative Analysis

Solution quality (Best / Mean):

HBALS obtained the lowest best-tour length among the implemented algorithms on all six tested instances. Compared with the implemented BA baseline, HBALS reduced the best-tour length by 15.3% to 68.0% across the six instances, with an arithmetic mean reduction of 41.5%. The magnitude of the reduction varied substantially across instances, with larger differences observed on kroA100, ch150, and rat195.

Stability (Std):

HBALS showed relatively small standard deviations across the six tested instances, indicating limited variation in the best solution quality obtained across independent runs. Its standard deviations ranged from 1.68 on eil51 to 79.06 on kroA100. The baseline algorithms exhibited different levels of run-to-run variation: GA and ACO showed comparatively larger standard deviations on several instances, while BA also exhibited non-zero variability under the stochastic configuration used in this study. PSO generally showed smaller standard deviations, although its solution quality differed substantially from HBALS. Overall, the standard-deviation results should be interpreted together with solution quality, since lower variability alone does not necessarily indicate better performance.

Statistical significance:

A Friedman test was conducted on the mean tour lengths across the six benchmark instances to examine whether the five algorithms exhibited statistically different performance. The test indicated a statistically significant difference among the algorithms $\chi^2 = 24.0$, $p < 0.001$. The mean ranks were 1.00 for HBALS, 2.00 for ACO, 3.00 for PSO, 4.00 for BA, and 5.00 for GA. Post-hoc Nemenyi analysis showed statistically significant differences between HBALS and BA $p = 0.009$ and between HBALS and GA $p = 0.0001$. The differences between HBALS and ACO $p = 0.809$ and between HBALS and PSO $p = 0.183$ were not statistically significant at the 0.05 level. Because the analysis is based on six benchmark instances, the results should be interpreted as evidence for differences among the tested algorithm configurations rather than as a universal conclusion about the underlying methods.

Runtime:

HBALS required more computation time than BA, GA, and PSO on all six instances, while ACO generally required more time than HBALS. This additional cost is associated with the neighborhood perturbation and local-search procedures used by HBALS.

6.5.4 Computational Cost

HBALS introduces additional computational overhead compared with the basic Bees Algorithm because selected solutions undergo neighborhood perturbation and local-search refinement. The measured runtime increases with instance size, ranging from 4.62 seconds on eil51 to 161.51 seconds on rat195 under the tested configuration. HBALS was slower than BA, GA, and PSO on all tested instances, but generally faster than ACO.

The runtime results demonstrate the computational cost of the hybrid search strategy. Because the present study does not include an ablation analysis, the individual contribution of each HBALS component to the observed solution quality and runtime cannot be isolated.

On the tested instances, HBALS generally approached its final solution quality within the early part of the 300-iteration budget (Figs. 1–6, Appendix A). The convergence behavior varies across instances, and the present study does not establish a universal convergence-rate advantage over other metaheuristics.

7. Discussion and Analysis

This section discusses the observed performance of HBALS on the selected TSP benchmark instances, the role of its hybrid components, its stability across independent runs, and its convergence behavior.

7.1 Algorithm Performance Analysis

The experimental results show that HBALS consistently obtained lower best-tour lengths than the implemented BA baseline across the six tested TSPLIB instances. The reduction in best-tour length relative to BA ranged from 15.3% to 68.0% across the tested instances, although the magnitude of the difference varied substantially with problem instance. The method combines mixed Nearest-Neighbor and random initialization with elite-site exploitation, selected-site exploration, adaptive neighborhood control, and local-search refinement.

The results are consistent with the intended role of these components: Nearest-Neighbor initialization provides a set of relatively structured starting solutions, while random initialization preserves additional diversity. Elite-site search increases exploitation around promising tours, whereas selected-site search provides a lighter form of exploration. The adaptive neighborhood mechanism is designed to change the perturbation intensity according to recent progress, although in the tested runs it settled at a single swap (see Section 6.2). Because the current study evaluates

the complete hybrid configuration without ablation experiments, the individual contribution of these components cannot be isolated.

7.2 Strengths and Weaknesses of the Algorithm

7.2.1 Strengths

Effective exploration–exploitation balance: the presence of scout bees provides additional exploration and may reduce the risk of premature convergence to local optima, while bees allocated to elite sites ensure precise exploitation.

Multi-point search capability: the ability to examine several regions of the solution space simultaneously increases the likelihood of finding better solutions.

7.2.2 Weaknesses

High sensitivity to parameters: parameters such as n_b, m, e, nep, nsp, and the neighborhood size ngh must be carefully tuned; otherwise, convergence speed and quality suffer.

Increased computational cost on larger tested instances: as the number of bees and sites increases, the number of objective-function evaluations per iteration grows substantially.

Risk of premature convergence in the base version: with poorly tuned parameters, the algorithm may stall around a single dominant region and mistake a local optimum for the final answer.

Greater memory and computation requirements in advanced variants: versions using 2-opt or Lightweight 3-opt-style refinement incur higher time cost and are more computationally demanding to run.

7.3 Effect of the Hybrid Components

HBALS outperformed the implemented BA baseline in terms of the best solution found on all six tested instances. However, the current experiments evaluate the complete hybrid configuration rather than individual components separately. Therefore, the observed improvement cannot be attributed exclusively to Nearest-Neighbor initialization, adaptive neighborhood control, 2-opt, or lightweight 3-opt-style refinement.

The local-search procedures increase exploitation around promising solutions, while the adaptive neighborhood mechanism is designed to change the perturbation intensity according to recent search progress. These mechanisms provide a plausible explanation for the observed behavior, but their individual effects should be investigated through ablation experiments in future work.

7.4 Stability and Convergence Analysis

7.4.1 Stability

Across the 20 independent runs, HBALS showed relatively small standard deviations on the tested instances. This indicates limited variation in the best solution quality obtained across runs. However, the observed stability is specific to the selected benchmark instances and parameter configuration and should not be generalized beyond the present experiments.

7.4.2 Convergence

On the tested instances, HBALS generally approached its final solution quality within the early part of the 300-iteration budget (Figs. 1–6, Appendix A). Because the neighborhood size settled at a single swap in the tested runs (see Section 6.2), this behavior should not be attributed to adaptive neighborhood control, and the present experiments do not isolate its effect from the other components of HBALS. The present experiments support this observed behavior, but they do not establish a universal convergence-rate advantage over other metaheuristics.

8. Comparison with Selected Recent Methods

Table 5 compares HBALS with selected recent Bees-Algorithm-based approaches for which results on overlapping TSPLIB instances are available. HBALS reaches the BKS on eil51, berlin52, and st70, while remaining relatively close to the BKS on the larger tested instances. The comparison with Sahin (2023) is mixed: the reported Sahin results are better on kroA100, ch150, and rat195, while HBALS obtains the BKS on several smaller instances. The comparison should be interpreted cautiously because published studies may use different implementations, parameter settings, computational environments, and stopping criteria.

Table 5. Comparison of HBALS with selected recent methods on benchmark TSP instances

Dataset	Algorithm	Best	Mean	Std	Mean Gap (%)	Time (s)
eil51 (BKS=426)	HBALS	426	427.85	1.68	0.43	4.62
	Sahin (2023) [3]	–	–	–	–	–
	Suluova et al. (2024) [2]	437	437	–	2.58	–
berlin52 (BKS=7542)	HBALS	7542	7543	2.00	0.01	5.31
	Sahin (2023) [3]	7544	7544	0	0.03	–
	Suluova et al. (2024) [2]	7930	7930	–	5.14	–
st70 (BKS=675)	HBALS	675	679.4	2.82	0.65	11.91
	Sahin (2023) [3]	677	677	0.04	0.31	–
	Suluova et al. (2024) [2]	696	696	–	3.11	–
kroA100 (BKS=21282)	HBALS	21296	21388.6	79.06	0.50	39.17
	Sahin (2023) [3]	21285	21285	0	0.02	–

	Suluova et al. (2024) [2]	22018	22018	–	3.46	–
ch150 (BKS=6528)	HBALS	6570	6627.95	40.13	1.53	83.68
	Sahin (2023) [3]	6552	6554	2.43	0.37	–
	Suluova et al. (2024) [2]	6928	6928	–	6.13	–
rat195 (BKS=2323)	HBALS	2349	2375.3	14.16	2.25	161.51
	Sahin (2023) [3]	2340	2352	7.61	0.73	–
	Suluova et al. (2024) [2]	–	–	–	–	–

Note: — indicates that the corresponding result was not reported in the original source. Values for Sahin (2023) [3] and Suluova et al. (2024) [2] are taken from the original publications and were not re-run.

8.1 Comparative Analysis by Dataset Size

Small datasets (eil51, berlin52)

On eil51, HBALS achieved the best-known solution (BKS = 426) exactly, with a mean of 427.85 and a mean gap of only 0.43%, demonstrating precise and stable performance. By comparison, Suluova et al. (2024) [2] recorded a mean of 437 and a 2.58% mean gap— a weaker result.

On berlin52, HBALS again reached the BKS with a mean gap of just 0.01%, the best of the three methods. Sahin (2023) [3] reported a 0.03% mean gap, while Suluova et al. (2024) [2] reported a 5.14% mean gap.

These results demonstrate HBALS's high precision and stable behavior on small-scale problems.

Medium datasets (st70, kroA100)

On st70, HBALS reached a best value of 675 with a 0.65% mean gap — a favorable result. Sahin (2023) [3] achieved a lower mean gap (0.31%) and ranked first, while Suluova et al. (2024) [2] performed worse, at 3.11%.

On kroA100, HBALS obtained a best value of 21296 and a mean value of 21388.60, corresponding to a mean gap of 0.50% from the BKS of 21282. Sahin (2023) [3] reported a best value and mean value of 21285, which are closer to the BKS. Suluova et al. (2024) [2] reported a mean gap of 3.46%. HBALS therefore provides a competitive result but does not achieve the best reported performance for this instance.

Large datasets (ch150, rat195)

On ch150, HBALS obtained a best value of 6570 and a mean value of 6627.95, corresponding to a mean gap of 1.53% from the BKS of 6528. Sahin (2023) [3] reported a best value of 6552 and a mean value of 6554, both of which are closer to the BKS. Suluova et al. (2024) [2] reported a mean gap of 6.13%. The result indicates that HBALS remains competitive on this larger tested instance, although the reported Sahin result is better.

On rat195, HBALS obtained a best value of 2349 and a mean value of 2375.30, corresponding to a mean gap of 2.25% from the BKS of 2323. Sahin (2023) [3] reported a best value of 2340 and a mean value of 2352, both of which are lower than the corresponding HBALS values. Suluova et al. (2024) [2] did not report a result for rat195. HBALS therefore remained competitive on this instance but did not outperform the reported Sahin result.

8.2 Overall Analysis and Key Findings

Across the six evaluated TSPLIB instances, HBALS reached the BKS on three instances and produced mean gaps of at most 2.25% from the BKS on the tested set. Compared with the implemented BA baseline, HBALS obtained lower best-tour lengths on all six instances. The runtime increased with instance size, reflecting the additional computational cost of neighborhood perturbation and local-search refinement. The comparison with selected recent studies shows that HBALS is competitive on several instances but does not consistently outperform all published methods.

9. Conclusion and Future Work

This study examined the Bees Algorithm and its improved variants, with particular focus on solving the Traveling Salesman Problem (TSP). The algorithm's structure, biological rationale, and search cycle were first described, showing how inspiration from the natural foraging behavior of bees produces a powerful mechanism for balancing exploration and exploitation in complex optimization problems. The TSP was then modeled in detail, and the adaptation of both BA and the proposed hybrid variant, HBALS, for obtaining high-quality TSP tours under practical computational budgets was explained.

9.1 Summary of Key Contributions

This study presented HBALS, a hybrid Bees Algorithm for the Traveling Salesman Problem that combines mixed Nearest-Neighbor and random initialization, elite- and selected-site search, adaptive neighborhood control, and lightweight local-search refinement. The method was evaluated on six TSPLIB instances using 20 independent runs and a maximum of 300 iterations.

Under the experimental configuration used in this study, HBALS reached the best-known solution on eil51, berlin52, and st70 and produced solutions within 2.25% of the BKS in terms of mean tour length on the tested instances. HBALS also obtained lower best-tour lengths than the implemented BA, ACO, GA, and PSO baselines on all six instances. Relative to the implemented BA baseline, the reduction in best-tour length ranged from 15.3% to 68.0%, with an arithmetic

mean reduction of 41.5% across the six instances. A Friedman test indicated statistically significant differences among the five tested algorithm configurations $\chi^2 = 24.0$, $p < 0.001$. Post-hoc Nemenyi analysis showed statistically significant differences between HBALS and BA $p = 0.009$ and between HBALS and GA $p = 0.0001$, while the differences between HBALS and ACO $p = 0.809$ and between HBALS and PSO $p = 0.183$ were not statistically significant at the 0.05 level. These results support the usefulness of the tested hybrid configuration, while broader benchmarking and ablation analysis are required before drawing more general conclusions.

9.2 Limitations of the Study

Limited effect of the adaptive neighborhood mechanism: because the neighborhood size is updated with integer truncation, it decreases from 3 to 1 within the first two iterations and cannot increase from 1 with β = 1.05. The perturbation strength therefore remains at one random swap for the rest of each run, and the adaptive expansion step is effectively inactive on the tested instances.

High dependence on parameter tuning: parameters such as n_b, m, e, nep, nsp, and patch size play a highly sensitive role in the algorithm's efficiency, and manually tuning them for each problem can be difficult and time-consuming.

Relatively high computational cost in hybrid variants: local-search procedures add computational overhead and may increase runtime as the problem size grows.

No guarantee of reaching the global optimum: like other metaheuristics, the Bees Algorithm's convergence is not guaranteed, and convergence to local optima remains possible, particularly in very high-dimensional problems.

Increasing computational requirements for larger TSP instances: as the number of cities increases, the search space and cost of evaluating and refining candidate tours also increase.

The experimental evaluation is limited to six TSPLIB instances with up to 195 cities and therefore does not establish the behavior of HBALS on substantially larger TSP instances. In addition, the current study evaluates the complete hybrid method without an ablation analysis; consequently, the individual contributions of Nearest-Neighbor initialization, adaptive neighborhood control, 2-opt, and lightweight 3-opt-style refinement cannot be isolated. Comparisons with published methods should also be interpreted cautiously because differences in implementation, parameter settings, computational resources, and stopping criteria may affect reported results.

9.3 Suggested Directions for Future Research

Developing self-adaptive or parameter-free variants: designing algorithms that dynamically adjust parameter values during execution, reducing dependence on manual tuning.

Combining the Bees Algorithm with machine learning techniques: using reinforcement learning or neural network models to learn more effective search patterns for complex problems.

Developing lower-complexity, higher-speed variants for large-scale TSP: using lighter local search or fast approximate variants to increase convergence speed.

Employing parallel and distributed versions of the algorithm: running the algorithm on multi-core processors or GPUs to increase efficiency on very large instances.

Combining BA with other algorithms such as PSO, GA, or ACO: building new hybrid variants that simultaneously leverage the advantages of multiple swarm-intelligence algorithms.

Extending the algorithm to more complex variants such as the multiple TSP (mTSP), dynamic TSP (DTSP), and Vehicle Routing Problem (VRP): applying new algorithmic adaptations to solve more realistic problems in transportation, logistics, and robot routing.

Statements and Declarations

Funding. No funding was received for conducting this study.

Competing interests. The author has no relevant financial or non-financial interests to disclose.

Author contributions. Erfan Khadiv, the sole author, was responsible for the conception and design of the study, the software implementation, the experiments, the analysis of the results, and the writing of the manuscript.

Ethics approval and consent. Not applicable. This study did not involve human participants or animals.

Data and code availability. The benchmark instances used in this study (eil51, berlin52, st70, kroA100, ch150, and rat195) are taken from the TSPLIB library [14].

The best-known solution (BKS) values reported in Table 2 are the optimal tour lengths published in the same library. Results for Sahin (2023) [3] and Suluova et al. (2024) [2] in Table 5 are taken from the original publications and were not re-run.

The source code, the instance files used, and all experimental results and convergence plots are available at https://github.com/ErfanKhadiv/HBALS-TSP.

Appendix A — Supplementary Experimental Results and Convergence Analysis

Figs. 1–6 illustrate the convergence behavior of HBALS across the six selected TSPLIB instances. Each curve represents the average best-so-far tour cost over 20 independent runs, with the shaded region showing the minimum–maximum range across runs.

Fig. 1 Convergence of HBALS on the eil51 TSPLIB instance over 20 independent runs

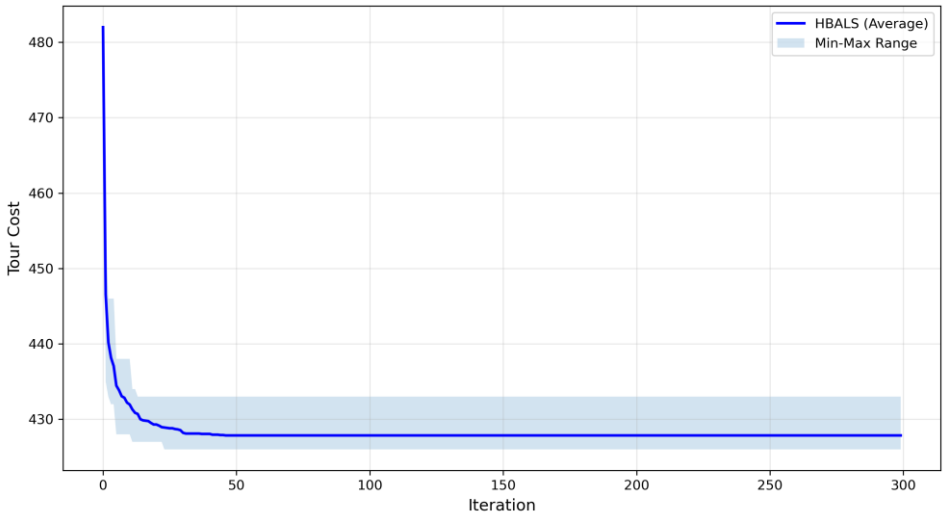


Fig. 2 Convergence of HBALS on the berlin52 TSPLIB instance over 20 independent runs

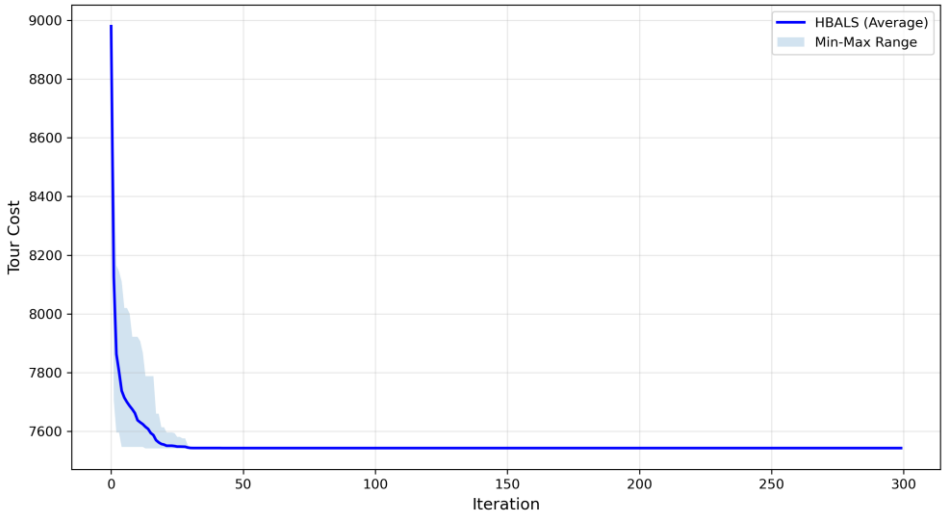


Fig. 3 Convergence of HBALS on the st70 TSPLIB instance over 20 independent runs

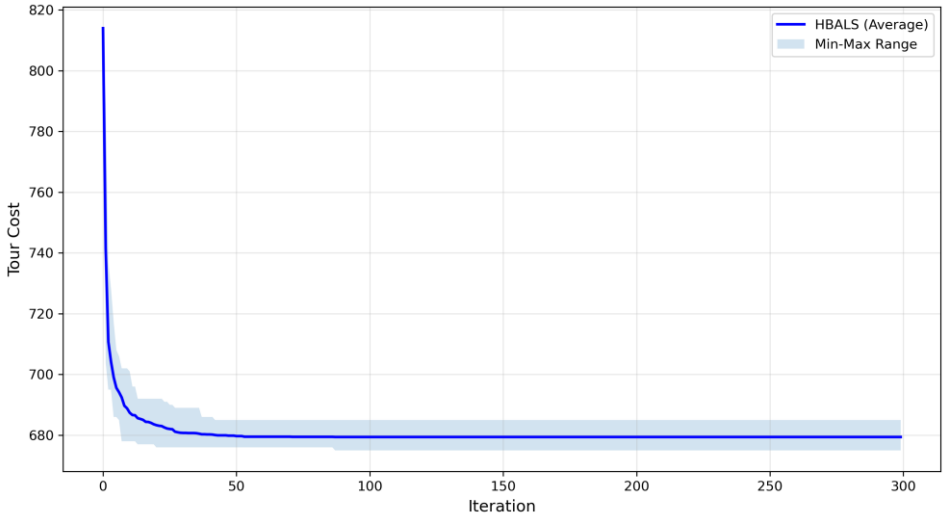


Fig. 4 Convergence of HBALS on the kroA100 TSPLIB instance over 20 independent runs

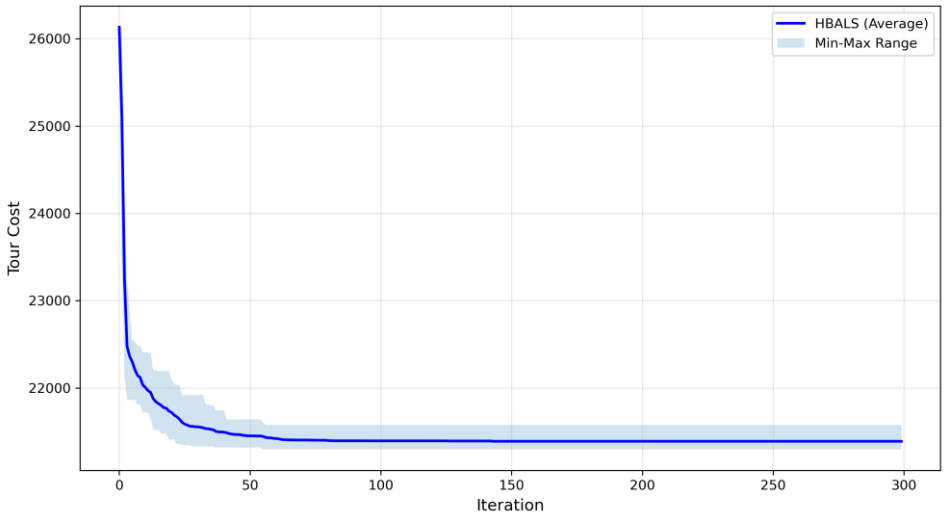


Fig. 5 Convergence of HBALS on the ch150 TSPLIB instance over 20 independent runs

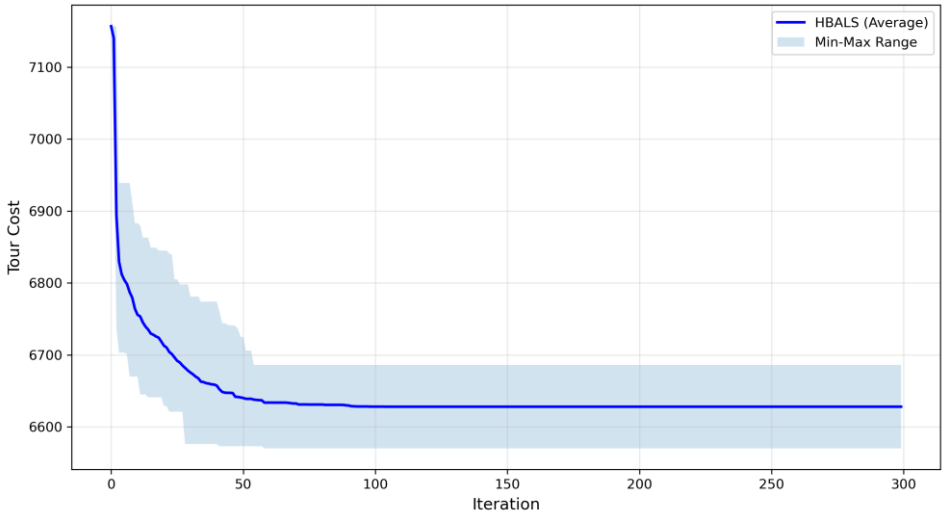
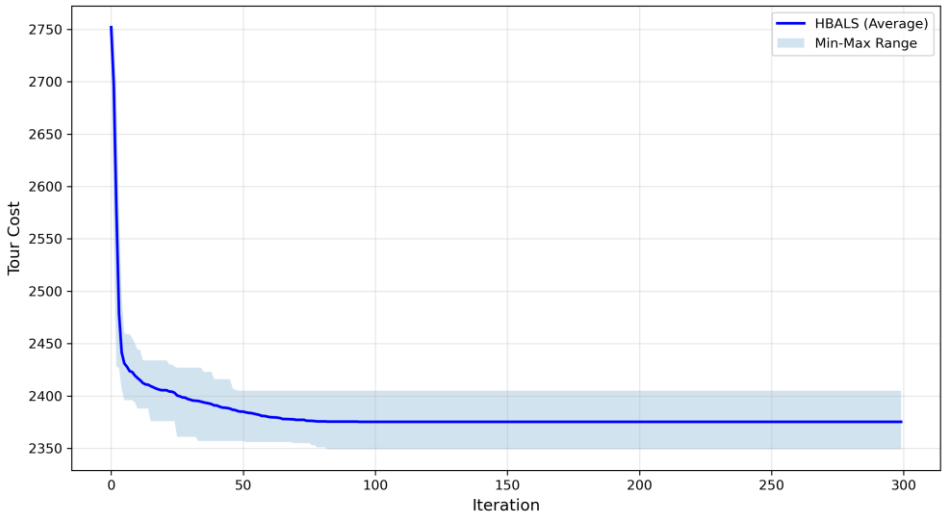


Fig. 6 Convergence of HBALS on the rat195 TSPLIB instance over 20 independent runs



References

[1] Blum, Christian & Roli, Andrea. (2003). Metaheuristics in Combinatorial Optimization: Overview and Conceptual Comparison. ACM Comput. Surv. 35. 268–308. https://doi.org/10.1145/937503.937505

[2] Suluova, H.F.; Pham, D.T. A New Single-Parameter Bees Algorithm. Biomimetics 2024, 9, 634. https://doi.org/10.3390/biomimetics9100634

[3] Sahin, M. Solving TSP by using combinatorial Bees algorithm with nearest neighbor method. Neural Comput & Applic 35, 1863–1879 (2023). https://doi.org/10.1007/s00521-022-07816-y

[4] Hartono, N., Suluova, H. F., Eker, F. M., Zeybek, S., & Caterino, M. (2024). A Comparative Study of Reduced Parameter Versions of the Bees Algorithm for Traveling Salesman Problem. Journal of Integrated System, 7(1), 1–12. https://doi.org/10.28932/jis.v7i1.8602

[5] Ismail, A. H., Ruslan, W., & Pham, D. T. (2023). A user-friendly Bees Algorithm for continuous and combinatorial optimisation. Cogent Engineering, 10:2, 2278257. https://doi.org/10.1080/23311916.2023.2278257

[6] Ismail, A. H. (2021). Enhancing the bees algorithm using the traplining metaphor. PhD Thesis, University of Birmingham. https://etheses.bham.ac.uk/id/eprint/12128/

[7] Seeley, T. D. (1995). The Wisdom of the Hive: The Social Physiology of Honey Bee Colonies. Cambridge, MA: Harvard University Press. https://doi.org/10.4159/9780674043404

[8] Camazine, S., Deneubourg, J.L., Franks, N.R., Sneyd, J., Theraulaz, G., & Bonabeau, E. (2001). Self-Organization in Biological Systems. Princeton University Press. https://doi.org/10.1515/9780691212920

[9] Pham, D. T., Castellani, M., Sholedolu, M., & Ghanbarzadeh, A. (2008). The Bees Algorithm and Mechanical Design Optimisation. In Proceedings of the 5th International Conference on Informatics in Control, Automation and Robotics (ICINCO-ICSO 2008), Funchal, Madeira, Portugal, pp. 250–255. https://www.scitepress.org/papers/2008/15061/15061.pdf

[10] Biesmeijer, J. & Seeley, T. (2005). The use of waggle dance information by honey bees throughout their foraging careers. Behavioral Ecology and Sociobiology, 59, 133–142. https://doi.org/10.1007/s00265-005-0019-6

[11] Yuce, B., Packianather, M., Mastrocinque, E., Pham, D., & Lambiase, A. (2013). Honey Bees Inspired Optimization Method: The Bees Algorithm. Insects, 4, 646–662. https://doi.org/10.3390/insects4040646

[12] Pop, P.C., Cosma, O., Sabo, C., & Sitar, C.P. (2024). A comprehensive survey on the generalized traveling salesman problem. European Journal of Operational Research, 314(3), 819–835. https://doi.org/10.1016/j.ejor.2023.07.022

[13] Hamza, A., Haj Darwish, A., & Rihawi, O. (2023). A new local search for the bees algorithm to optimize multiple traveling salesman problem. Intelligent Systems with Applications, 18, 200242. https://doi.org/10.1016/j.iswa.2023.200242

[14] Reinelt, G. (1991). TSPLIB: A traveling salesman problem library. ORSA Journal on Computing, 3(4), 376–384. https://doi.org/10.1287/ijoc.3.4.376. Library available at https://comopt.ifi.uni-heidelberg.de/software/TSPLIB95/